

■ Complete Python Learning Roadmap

Basic → Intermediate → Advanced → Real-World Projects

Goal: Learn Python from fundamentals to professional/advanced level.

LEVEL 1 — BASIC PYTHON

■ 1. Python Fundamentals

- Python introduction
- Python syntax
- Variables
- Naming conventions
- input()
- Python installation & setup
- Comments
- Constants
- print()
- Basic program structure

■ 2. Data Types

- int
- complex
- bool
- type()
- Mutable vs Immutable
- float
- str
- None
- Type conversion / casting

■ 3. Strings

- Creating strings
- Negative indexing
- String methods
- Unicode
- chr()
- f-strings
- Indexing
- Slicing
- Escape characters
- ord()
- String formatting
- .format()

■ 4. Operators

- Arithmetic
- Assignment
- Membership
- Bitwise
- Comparison
- Logical
- Identity
- Operator precedence

■ 5. Conditional Statements

- if
- if-elif-else
- Ternary operator
- if-else
- Nested if
- match-case

■ 6. Loops

- for
- range()
- break
- pass
- while-else
- while
- Nested loops
- continue
- for-else

■ 7. Lists

- Creating lists
- Slicing
- Indexing
- Updating

- Adding elements
- List methods
- List copying

■ 8. Tuples

- Creating tuples
- Slicing
- Tuple unpacking

■ 9. Sets

- Creating sets
- Set methods
- Intersection
- Symmetric difference

■ 10. Dictionaries

- Creating dictionaries
- Accessing values
- Removing
- Nested dictionaries

■ 11. Functions

- Creating functions
- Parameters
- Return value
- Keyword arguments
- *args
- Scope

■ 12. Modules & Packages

- import
- Built-in modules
- Packages

■ 13. Exception Handling

- Errors vs exceptions
- except
- finally
- Custom exceptions

- Removing elements
- Nested lists

- Indexing
- Tuple methods
- Nested tuples

- Adding/removing elements
- Union
- Difference
- Set operations

- Keys & values
- Adding/updating
- Dictionary methods
- Dictionary iteration

- Calling functions
- Arguments
- Default arguments
- Positional arguments
- **kwargs
- Lambda functions

- from ... import
- Creating your own module
- pip basics

- try
- else
- raise

LEVEL 2 — INTERMEDIATE PYTHON

■ 14. Comprehensions

- List comprehension
- Dictionary comprehension
- Nested comprehension
- Set comprehension
- Conditional comprehension

■ 15. Functional Programming

- Lambda
- filter()
- zip()
- map()
- reduce()
- enumerate()

■ 16. Advanced Functions

- First-class functions
- Nested functions
- Decorators

■ 17. Iterators & Generators

- Iterable vs iterator
- next()
- yield

■ 18. File Handling

- Opening files
- Writing
- with
- Text files

■ 19. Working with Data

- JSON
- pickle
- Deserialization

■ 20. Regular Expressions

- re
- search()
- findall()
- Groups
- Quantifiers

■ 21. Object-Oriented Programming

- Classes
- Attributes
- `__init__()`
- Class variables
- Class methods

■ 22. OOP Advanced

- Encapsulation
- Multiple inheritance
- Polymorphism
- `super()`
- Abstract classes

■ 23. Python Special Methods

- `__str__()`
- `__len__()`
- `__add__()`
- Other dunder methods

■ 24. Python Standard Library

- math
- datetime
- sys

- Higher-order functions
- Closures
- Function annotations

- `iter()`
- Generator functions
- Generator expressions

- Reading
- Appending
- File modes
- Binary files

- CSV
- Serialization

- Pattern matching
- `match()`
- `sub()`
- Character classes

- Objects
- Methods
- Instance variables
- Instance methods
- Static methods

- Inheritance
- Multilevel inheritance
- Method overriding
- Abstraction

- `__repr__()`
- `__eq__()`
- `__getitem__()`

- random
- os
- pathlib

- collections
- functools

- itertools
- statistics

■ 25. Environment & Package Management

- pip
- venv
- requirements.txt

- Virtual environments
- Package installation
- Environment variables

■ 26. Type Hints

- Type annotations
- dict
- Optional
- Any
- typing

- list
- tuple
- Union
- Generic types

■ 27. Testing & Debugging

- Debugging
- unittest
- Test cases
- Mocking basics

- Assertions
- pytest
- Fixtures

LEVEL 3 — ADVANCED PYTHON

■ 28. Advanced OOP

- SOLID principles
- Aggregation
- Protocols
- Descriptors

- Composition
- Abstract Base Classes
- Properties
- Metaclasses

■ 29. Advanced Decorators & Closures

- Decorator chaining
- Class decorators

- Parameterized decorators
- Advanced closures

■ 30. Context Managers

- with
- `__exit__()`
- Custom context managers

- `__enter__()`
- contextlib

■ 31. Concurrency

- Concurrency vs parallelism
- Processes
- Locks
- Thread pools

- Threads
- Thread synchronization
- Queues

■ 32. Multiprocessing

- multiprocessing
- Process pools

- Processes
- Inter-process communication

■ 33. Async Python

- async
- Coroutines
- Tasks

- await
- asyncio
- Futures

- Async HTTP concepts

■ 34. Memory & Performance

- Python memory model
- Garbage collection
- Profiling
- cProfile
- References
- Reference counting
- timeit
- Optimization

■ 35. Advanced Data Structures & Algorithms

- Stack
- Deque
- Trees
- Hash tables
- Sorting
- Big-O complexity
- Queue
- Linked list
- Graphs
- Searching
- Recursion

■ 36. Database Programming

- SQL fundamentals
- Python + SQLite
- CRUD
- Parameterized queries
- SQLite
- MySQL/PostgreSQL concepts
- Transactions
- Connection management

■ 37. Web Development

- HTTP basics
- Flask
- Routing
- JSON APIs
- REST APIs
- FastAPI
- Request/Response
- Authentication basics

■ 38. API Development & Consumption

- REST API
- API authentication
- Parameters
- Building APIs
- requests
- Headers
- Error handling

■ 39. Logging & Production Code

- logging
- Handlers
- Configuration
- Log levels
- Formatters
- Error monitoring concepts

■ 40. Packaging & Distribution

- Project structure
- Building packages
- Dependency management
- pyproject.toml
- Publishing packages
- Versioning

■ 41. Security

- Input validation
- Secrets management
- Secure API practices
- Password hashing concepts
- SQL injection prevention
- Environment variables

■ 42. Professional Python

- Clean code
- Documentation
- PEP 8
- Type-safe code

- Design patterns
- Code review

- Git/GitHub workflow
- Project architecture

LEVEL 4 — REAL-WORLD PROJECTS

■ Beginner Projects

- Calculator
- To-do list
- Contact book
- Number guessing game
- Student management system
- Quiz application

■ Intermediate Projects

- Expense tracker
- Password manager
- Web scraper
- File organizer
- Weather/API application
- SQLite CRUD application

■ Advanced Projects

- REST API
- E-commerce backend
- Multi-user database application
- Authentication system
- Async API service
- Production-style Python application

Progress Tracker

Basic: ■ Intermediate: ■ Advanced: ■ Projects: ■

Current progress: Variables/Data Types → Strings → String Methods → Operators. Next: Conditional Statements.